

Training a detector for strawberries, plants and glass

Handover document. Everything needed to train the perception model for the autonomous greenhouse harvester: what the classes are, what data to collect, where to get it, the complete pipeline source, and how the result will be judged.

Why this exists

The detector currently running on the robot thresholds redness by channel ratio. Measured against the 127 known berry positions in the simulator it reaches **69% recall**, and **66 of its 111** map estimates match no real berry. The harvest phase spends **39% of its time** aligning on things that are not fruit.

It cannot be tuned out of that. Its test is a ratio, $G < 0.33R$ and $B < 0.33R$, which is invariant to illumination *scale* - that is why it survives shade - but not to illumination *colour*. Late-afternoon light through glass lowers G/R for the entire scene, and at that point every surface passes the fruit test. A learned model fixes the class of problem rather than the instance, but only if it is trained on data that contains the failure.

1. The four classes

Class	id	Annotation	What it is for	Priority
ripe_strawberry	0	bounding box	The harvest target. The only class the mission acts on. Small objects: a berry at 2 m is about 15 px in a 640 px frame.	1
unripe_strawberry	1	bounding box	Not a target - a HARD NEGATIVE. Ripeness is a continuum, not a switch. A model trained only on ripe fruit learns 'roundish object on a plant' and reports every green berry. Its own class forces the boundary to be about colour maturity instead of shape.	2
strawberry_plant	2	box, or mask if affordable	Locates the growing row, and constrains where a berry may plausibly be: a 'berry' detected over bare floor is wrong. Easy to detect, hard to box tightly - foliage has no clean boundary.	3
glass	3	SEGMENTATION mask	The sim-to-real risk: a 2D lidar at glancing incidence on a pane returns nothing, so the wall is invisible. Read the warning in section 4 before investing here.	4

Class ids are fixed in `ml/classes.yaml` and are read by every script and by the robot-side scorer. Append new classes at the end; **never reorder**, or every label file ever written is silently remapped.

2. What kind of data

2.1 Format

YOLO throughout: one .txt per image, lines of `class cx cy w h`, all coordinates normalised to [0, 1].

An **empty file** means 'nothing here, and I am sure'. A **missing file** means 'unlabelled'. They are not the same, and confusing them inflates the false-negative count silently. Images with no fruit must get empty label files, not no file.

2.2 How much, and of what

Volume matters less than coverage of the conditions that break the current detector. 500 images spanning a day beat 5000 taken in one hour.

Condition to cover	Why it matters	Rough share
Morning, midday, late afternoon, overcast	The colour-temperature failure. This is the single most important axis.	4 blocks, balanced
Both aisle sides (sunlit row, shaded row)	Exposure extremes within one pass.	50 / 50
Ripe, half-ripe and green fruit	Ripeness is a continuum; the boundary cases are where the model will be wrong.	all three present

Condition to cover	Why it matters	Rough share
Fruit occluded by leaves	The majority case in reality. A model trained on clean fruit reports clean fruit only.	at least 40% of berries
Red objects that are NOT fruit: pipes, tools, crates, clothing	This is where false positives come from. They must be in the training set to be rejected.	at least 15% of frames
Frames with no fruit at all	Teaches 'nothing here'. Kept as empty label files.	about 30% of frames
Wet fruit, specular highlights	A white glare splits one berry into two blobs.	some
Range 0.3 m to 2.5 m	Small-object recall at range.	spread evenly

First target: 1500 to 3000 REAL annotated frames - public datasets, plus photographs of the actual greenhouse when they can be obtained. Rendered frames from the simulator are an optional supplement on top of that, never the base.

3. Where to get the data

Real images are the training set. The simulator is a supplement. That order is not a preference. The model has to work in a real greenhouse in real daylight; a model trained on rendered images learns the renderer's shading, the renderer's textures and its perfectly clean unoccluded fruit, and none of those exist on site. Start from real photographs and add renders only to fill conditions the real set does not contain.

3.1 Public datasets - real images, today, no site access

Names and descriptions below come from published work. **Verify the current URL and the licence before use** - hosting and terms change, and several are research-only. You download them; the conversion is already written (section 3.2).

Dataset	What it is	Notes
StrawDI_Db1	About 3100 real strawberry images from fields in Huelva, Spain, with instance segmentation masks.	Best single starting point. Masks convert to boxes trivially. Research use.
Roboflow Universe (search 'strawberry ripeness', 'strawberry detection')	Dozens of community sets, many already split into ripe / unripe.	Exports directly in YOLO format. Quality varies a lot - inspect before trusting. Check each licence.
DeepFruits (Sa et al., Sensors, 2016)	Seven fruit types including strawberry, bounding boxes.	Small but well documented.
Kaggle (search 'strawberry detection')	Several sets, mixed quality.	Same caveat as Roboflow.
GDD - Mei et al., 'Don't Hit Me! Glass Detection in Real-world Scenes', CVPR 2020	About 3900 images with glass region masks.	The reference dataset for glass.
Trans10K - Xie et al., ECCV 2020	About 10400 images of transparent objects, two categories.	Larger, broader than glass alone.

For plants: the strawberry field datasets above already contain plants - annotate the plant class on those same images rather than hunting for a separate source. CVPPP has leaf segmentation data if more is wanted; PlantVillage is disease-focused and shot on plain backgrounds, which is poor context for this problem.

3.2 Making a downloaded dataset usable: `import_real.py`

A list of dataset names is not a path. Every public set ships in its own format and numbers its classes from 0 with its own meanings: StrawDI gives per-instance **PNG masks**, Roboflow gives YOLO boxes with *its* class ids, others give COCO JSON or Pascal VOC XML. `import_real.py` converts all four into our layout and our class ids, so the day normally spent writing converters is already spent.

Always start with `--inspect`. The first real problem with a download is working out what it actually is - the archive rarely matches its description. It reports the format it detected, the image and label counts, the source class names when the set carries them, and prints the exact import command to run next.

```
# 0. What did I actually download?
python3 ml/import_real.py --inspect ~/downloads/StrawDI_Db1

# StrawDI-style: images + instance masks, NO ripeness information
python3 ml/import_real.py --src ~/downloads/StrawDI_Db1 --from strawdi \
  --default-class ripe_strawberry --out ~/real/strawdi

# Roboflow / Kaggle YOLO export: its class names -> ours
python3 ml/import_real.py --src ~/downloads/rf_strawberry --from yolo \
  --map "strawberry=ripe_strawberry,unripe=unripe_strawberry" \
  --out ~/real/roboflow

# COCO JSON and Pascal VOC exports take the same --map
python3 ml/import_real.py --src ~/downloads/set_coco --from coco \
  --map "..." --out ~/real/coco
```

Two rules the tool enforces rather than guesses:

- **`--map` is required** for every format that carries class names, and any source class not mapped is **dropped and reported**, never silently folded into class 0. A set whose 'strawberry' means any strawberry will poison

unripe_strawberry, which exists precisely as a hard negative.

- **--default-class is required** for mask datasets, because they do not label ripeness - a mask is a strawberry, full stop. Which class they all land in is a decision, not a default, and the tool warns when everything lands in ripe_strawberry so you know what the model is being taught.

Output is <out>/images and <out>/labels in our class ids, handed straight to prepare_dataset.py as a real source.

3.3 Real photographs of the actual greenhouse - the ones that matter most

Site access is not required to obtain these. Send this protocol to someone on site:

Take 200 photos with a phone, held about 0.8 m above the floor (the robot camera height), from the walking aisle, aimed at the plant row. Repeat the same 10 positions at 4 different times of day, including late afternoon. Include 20 photos with tools, pipes or a coloured jacket in frame. Do not tidy the scene - the mess is the point. Note the time and the weather for each block.

Annotate them with CVAT, Label Studio or Roboflow (all free at this size) and use them for fine-tuning and, crucially, as the **test set**. A score measured on rendered images does not predict a real greenhouse.

3.4 The digital twin - a supplement, not the training set

Once the real data is in, the simulator fills gaps in it. Its one advantage is perfect ground truth: the world knows where all 127 berries are, so it writes its own labels, thousands of them, overnight.

```
ros2 launch youbot_gazebo gazebo.launch.py
ros2 run youbot_slam dataset_capture --ros-args \
  -p session:=twin_morning -p target_frames:=2000

# record the conditions of each session - half the value of the set
ros2 topic pub -1 /dataset/conditions std_msgs/String \
  "{data: 'warm light, sun low from the west, north row shaded'}"
```

Labels never come from the colour detector. They are projected from the ground-truth berry catalogue through youbot_slam.lib.berry_view, the same visibility model the scorer uses. Annotating with the detector would teach its replacement to reproduce its two-in-three false positives exactly.

Use it for what real data cannot give: extreme viewpoints, conditions no photographs exist for, and frames where the berry count has to be exactly right. Do not let it become the bulk of the training set, and **never** put it in the test set - prepare_dataset.py refuses that outright.

4. The glass warning - read before investing

Glass detection from RGB will not make the robot safe. Glass has almost no appearance of its own: a model detects it from reflections, edge cues and the discontinuity between what is in front and behind, all of which change completely with viewpoint and lighting. Published glass segmenters report IoU in the 0.7 to 0.85 range on their own benchmarks and degrade sharply out of domain. That is a research result, not a protective device.

For the robot's actual problem - do not drive into the pane - the engineering answer, in order:

- **Mark the glass physically.** An opaque band at lidar height (0.20 m) along every pane the robot can approach. Costs a roll of tape, works every time, and is what commissioning stage 3 will recommend.
- **Add a sensor that sees glass.** Ultrasonic rangefinders are cheap and reflect off glass, which lidar does not.
- **Bound the workspace in the map,** so the planner never routes within reach of an unmarked pane.

Train the glass class as a **secondary cue and a report contribution** - it is legitimate and interesting work. Do not put it on the safety path.

5. The workflow

```
# Install in a virtualenv, NOT into the ROS Python environment:
# ultralytics pulls a torch build that will fight with ROS packages.
python3 -m venv ~/venv-ml && source ~/venv-ml/bin/activate
pip install -r ml/requirements.txt

# 0. Convert the REAL datasets you downloaded (see section 3.2).
python3 ml/import_real.py --inspect ~/downloads/StrawDI_Db1
python3 ml/import_real.py --src ~/downloads/StrawDI_Db1 --from strawdi \
    --default-class ripe_strawberry --out ~/real/strawdi

# 1. Merge every source into one YOLO tree with honest splits.
# REAL sources first - they are the training set.
python3 ml/prepare_dataset.py \
    --source ~/real/strawdi:real \
    --source ~/real/roboflow:real \
    --source ~/photos_serre_annotated:real \
    --source ~/youbot_datasets/twin_morning:synthetic \
    --out ~/strawberry_ds

# 2. Widen the daylight range offline (recommended - see section 6).
python3 ml/augment_daylight.py --dataset ~/strawberry_ds --split train

# 3. Train.
python3 ml/train.py --data ~/strawberry_ds/data.yaml \
    --model yolov8s --imgsz 960

# 4. Turn predictions into label files the robot-side scorer reads.
python3 ml/predict_to_labels.py \
    --weights runs/detect/strawberry/weights/best.pt \
    --images ~/strawberry_ds/test/images --out ~/preds/yolo

# 5. The SAME script runs the current colour threshold, so the two
# are compared on identical images by identical code.
python3 ml/predict_to_labels.py --baseline-colour \
    --images ~/strawberry_ds/test/images --out ~/preds/colour

# 6. Score both.
ros2 run youbot_commissioning stage8_detector -- \
    --labels ~/strawberry_ds/test/labels \
    --predict ~/preds/yolo \
    --baseline ~/preds/colour
```

Step 6 is the point of the whole exercise. **A model that does not beat the colour threshold on the same images has proved nothing.** Validation mAP from step 3 is for picking a checkpoint, not for reporting.

What prepare_dataset.py refuses to do

- **Split by image.** It splits by SOURCE GROUP. A robot at 0.2 m/s photographs the same plant twenty times; splitting those at random puts near-duplicates in train and val, the validation score climbs, and it measures nothing.

- **Put synthetic frames in the test set.** The test split is drawn from real sources only, and the script exits with an error if every source is synthetic.
- **Concatenate class ids blindly.** Every public dataset numbers its classes from 0; a strawberry set's class 0 and a glass set's class 0 mean different things. A per-source remap is mandatory.

6. The two settings that matter more than the model size

6.1 Image size: use 960 or 1280, never 640

Strawberries are small objects. A 34 mm berry at 2 m subtends about 15 px in a 640 px frame with this camera's 1.2 rad field of view, and the coarsest YOLO head has stride 32 - below roughly 20 px, recall falls off a cliff. Training at 960 px moves recall at range more than going from yolov8n to yolov8m does. It is the first thing to raise if distant fruit is being missed; do not blame the model size for it.

6.2 Hue augmentation: the default is wrong for this problem

Ultralytics defaults to `hsv_h = 0.015`, a very narrow band of colour shift. `ml/train.py` raises it to 0.05, because the exact failure being fixed *is* a colour shift. Training with the default hue range on midday images reproduces the colour threshold's bug inside a neural network - a more expensive way to get the same failure.

`augment_daylight.py` goes further because a hue rotation is generic and this failure is not. What breaks the detector is a one-directional blackbody shift: sunlight through glass going warm, every surface moving the same way at once, until the G/R test cannot tell a warm scene from a red object. The script produces copies of the training set from overcast 7500 K to sunset 3000 K, gamma-correct so the shift lands across the tone range instead of piling up in the shadows. **Labels are copied verbatim: changing the illuminant moves no geometry, which is why this is free.**

Sanity check before training: open a few `*_ct_sunset_3000K` images. If the plants now look red to you, that is exactly the condition the colour threshold fails in - and the model has now seen it.

7. How the result will be judged

Metric	Target	Why
Precision	at least 0.85	The current detector produces roughly three spurious estimates out of five. Precision is the metric that recovers the 39% of harvest time lost to aligning on nothing.
Recall	at least 0.80	Current colour threshold: about 0.69, measured over all 127 berries including ones never in view, so the true figure is somewhat better.
F1 against the colour baseline	strictly greater	Measured on identical images by identical code. Without this the comparison proves nothing.
Annotated test images	at least 100	Real images only. A score on renders does not predict a greenhouse.
Missing prediction files	zero	A missing file silently becomes 'detected nothing' and inflates the false-negative count.

8. Source

Verbatim from the repository. This document is generated by `ml/make_handover_pdf.py` reading these files, so the listings are by construction the code that is in the tree. Regenerate after any change.

`ml/classes.yaml`

59 lines

```
# =====
# Class definitions -- the single source of truth for the whole pipeline.
# =====
# Every script here, and stage8_detector in youbot_commissioning, reads
# these ids. Changing the ORDER silently remaps every label file ever
# written, so append at the end and never reorder.
# =====

names:
  0: ripe_strawberry
  1: unripe_strawberry
  2: strawberry_plant
  3: glass

# -----
# What each class is for, and how hard it is. Read this before spending
# a week annotating the wrong thing.
# -----
notes:

ripe_strawberry:
  task: detection (bounding box)
  why: the harvest target. This is the only class the mission acts on.
  difficulty: moderate. Small objects -- a berry at 2 m is ~15 px in a
    640 px frame -- so train at imgsiz 960 or higher or recall collapses
    at range.
  priority: 1

unripe_strawberry:
  task: detection (bounding box)
  why: >
    Not a harvest target, but essential as a HARD NEGATIVE. Ripeness is a
    continuum, not a switch; a model trained only on ripe fruit learns
    "roundish object on a plant" and reports every green berry. Giving
    unripe fruit its own class forces the decision boundary to be about
    colour maturity instead of shape.
  difficulty: moderate, plus genuine label noise at the boundary. Write
    down your ripeness rule BEFORE annotating and stick to it.
  priority: 2

strawberry_plant:
  task: detection (box) or segmentation (mask)
  why: >
    Locates the growing row. Useful for two things: constraining where a
    berry may plausibly be (a "berry" detected over bare floor is wrong),
    and as context for the mission's approach.
  difficulty: easy to detect, hard to box tightly -- foliage has no clean
    boundary. If you can afford masks, this class is where they pay off.
  priority: 3

glass:
  task: SEGMENTATION, not detection
  why: >
    The sim-to-real risk identified in commissioning stage 3: a 2D lidar
    at glancing incidence on a pane gets no return, so the wall is
    invisible and the robot drives into it.
  difficulty: HARD, and read the warning below before investing in it.
  priority: 4
```

`ml/requirements.txt`

20 lines

```
# Offline training only -- none of this is needed on the robot.
#
# Install in a virtualenv, NOT into the ROS Python environment: ultralytics
# pulls a torch build that will fight with whatever ROS packages expect.
# python3 -m venv ~/venv-ml && source ~/venv-ml/bin/activate
# pip install -r requirements.txt

ultralytics>=8.2          # YOLOv8/v11 training, validation, export
torch>=2.0                # pulled by ultralytics; pin here if you need CUDA
                           # a specific way (see pytorch.org for the index URL)
pillow>=10.0              # image IO
```

```

numpy>=1.24
pyyaml>=6.0
scipy>=1.10

# data.yaml / classes.yaml
# connected components for the colour baseline
# (predict_to_labels.py falls back without it)

# Optional, useful:
# matplotlib          # plotting curves yourself
# onnx, onnxruntime    # if you export for deployment

```

ml/import_real.py

503 lines

```

#!/usr/bin/env python3
"""import_real -- turn a downloaded real-image dataset into our YOLO layout.

WHY THIS EXISTS

The plan says "train on real strawberries", and the README lists where to get
them. A list of dataset names is not a path: every public set ships in a
different format and numbers its classes from 0 with its own meanings, so
"download StrawDI" is followed by a day of writing converters. This is that
day, done once.

Four input formats cover essentially everything you will find:

    strawdi  images + per-instance PNG masks    (StrawDI_Db1 and similar)
    yolo     images + .txt boxes + data.yaml     (Roboflow, most Kaggle sets)
    coco     images + one annotations.json       (COCO-style exports)
    voc      images + one .xml per image         (Pascal VOC style)

Output is always the same: <out>/images and <out>/labels in OUR class ids from
classes.yaml, ready for prepare_dataset.py as a `real` source.

START WITH --inspect. The first real problem with a downloaded dataset is
working out what it actually is; the archive rarely matches its description.

    python3 ml/import_real.py --inspect ~/downloads/StrawDI_Db1

THE CLASS MAPPING IS NOT OPTIONAL AND NOT GUESSABLE

A strawberry set's class 0 and a glass set's class 0 mean different things.
Worse, a strawberry set's "strawberry" may mean "ripe strawberry" or "any
strawberry", and those are different classes for us -- the second one poisons
the unripe class, which exists specifically as a hard negative. So the mapping
is stated explicitly on the command line, `--map` is required for every format
that carries class names, and unmapped classes are DROPPED loudly rather than
silently folded into class 0.

RIPENESS IN MASK DATASETS

StrawDI and most segmentation sets do not label ripeness at all: a mask is a
strawberry, full stop. Everything therefore lands in one class, and which one
is your decision, not the importer's. `--default-class` makes you say it, and
the tool warns that a set imported as all-ripe will teach the model that green
berries are ripe if the set contains any.
"""

from __future__ import annotations

import argparse
import json
import os
import shutil
import sys
from collections import Counter

import yaml

IMG_EXT = (".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff")

# ----- utils
def load_our_classes(path: str) -> dict:
    with open(path) as fh:
        return {int(k): v for k, v in yaml.safe_load(fh)["names"].items()}

def parse_map(spec: str | None, ours: dict) -> dict | None:
    """`--map "strawberry=0,unripe=ripe_strawberry" -> {source: our_id}."""
    if not spec:
        return None
    by_name = {v: k for k, v in ours.items()}
    out = {}
    for pair in spec.split(","):
        if "=" not in pair:

```

```

        raise SystemExit(f"--map entry {pair!r} needs the form src=dest")
    src, dest = (p.strip() for p in pair.split("=", 1))
    if dest.isdigit():
        dest_id = int(dest)
    elif dest in by_name:
        dest_id = by_name[dest]
    else:
        raise SystemExit(
            f"--map destination {dest!r} is not one of our classes: "
            f"{sorted(by_name)}")
    if dest_id not in ours:
        raise SystemExit(f"--map destination id {dest_id} is not in "
            f"classes.yaml ({sorted(ours)}")
    out[src] = dest_id
    return out

def find_images(root: str) -> list[str]:
    hits = []
    for base, _dirs, files in os.walk(root):
        for f in files:
            if f.lower().endswith(IMG_EXT):
                hits.append(os.path.join(base, f))
    return sorted(hits)

def stem(path: str) -> str:
    return os.path.splitext(os.path.basename(path))[0]

def need_numpy():
    try:
        import numpy as np
        from PIL import Image
        return np, Image
    except ImportError:
        raise SystemExit("mask import needs pillow and numpy: "
            "pip install -r ml/requirements.txt")

class Writer:
    """Writes the output tree and keeps the tally."""

    def __init__(self, out: str, prefix: str):
        self.img_dir = os.path.join(out, "images")
        self.lab_dir = os.path.join(out, "labels")
        os.makedirs(self.img_dir, exist_ok=True)
        os.makedirs(self.lab_dir, exist_ok=True)
        self.prefix = prefix
        self.n_img = 0
        self.n_box = 0
        self.n_empty = 0
        self.per_class = Counter()
        self.dropped = Counter()

    def add(self, image_path: str, boxes) -> None:
        name = f"{self.prefix}_{stem(image_path)}"
        ext = os.path.splitext(image_path)[1]
        shutil.copy2(image_path, os.path.join(self.img_dir, name + ext))
        with open(os.path.join(self.lab_dir, name + ".txt"), "w") as fh:
            for cls, cx, cy, w, h in boxes:
                fh.write(f"{cls} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}\n")
                self.per_class[cls] += 1
        self.n_img += 1
        self.n_box += len(boxes)
        if not boxes:
            self.n_empty += 1

# ----- inspect
def inspect(src: str) -> int:
    print(f"inspecting {src}\n")
    if not os.path.isdir(src):
        print(" not a directory")
        return 2
    imgs = find_images(src)
    txts, xmls, jsons, yamls, masks = [], [], [], [], []
    for base, _d, files in os.walk(src):
        for f in files:
            p = os.path.join(base, f)
            low = f.lower()
            if low.endswith(".txt"):
                txts.append(p)
            elif low.endswith(".xml"):
                xmls.append(p)
            elif low.endswith(".json"):

```

```

        jsons.append(p)
    elif low.endswith(".yaml", ".yml"):
        yamls.append(p)
# A "label"/"mask"/"gt" directory of images alongside an image directory
# is the segmentation signature.
for p in imgs:
    parts = {q.lower() for q in p.split(os.sep)}
    if parts & {"label", "labels", "mask", "masks", "gt", "annotations"}:
        masks.append(p)

print(f"  images : {len(imgs)}")
print(f"  of which in a label/mask directory : {len(masks)}")
print(f"  .txt    : {len(txts)}")
print(f"  .xml    : {len(xmls)}")
print(f"  .json   : {len(jsons)}")
print(f"  .yaml   : {len(yamls)}")

for y in yamls[:3]:
    try:
        d = yaml.safe_load(open(y)) or {}
    except Exception:
        continue
    if "names" in d:
        print(f"\n {os.path.relpath(y, src)} declares classes:")
        names = d["names"]
        items = (names.items() if isinstance(names, dict)
                  else enumerate(names))
        for k, v in items:
            print(f"    {k}: {v}")
        print("\n -> looks like YOLO. Use:")
        print("    --from yolo --map \"\" +
              ", ".join(f"{v}=<our class>" for _k, v in
                        (names.items() if isinstance(names, dict)
                         else enumerate(names))) + "\"")
        return 0

if masks and len(masks) * 3 > len(imgs):
    print("\n -> looks like per-instance MASKS (StrawDI style). Use:")
    print("    --from strawdi --default-class ripe_strawberry")
    return 0
if jsons:
    print("\n -> a .json is present; if it has 'annotations' and "
          "'categories' it is COCO. Use --from coco --map ...")
    return 0
if xmls:
    print("\n -> one .xml per image: Pascal VOC. Use --from voc --map ...")
    return 0
if txts:
    print("\n -> .txt beside images with no data.yaml: probably YOLO "
          "with undocumented class ids. Open one and see how many "
          "distinct first fields there are, then --from yolo --map ...")
    return 0
print("\n -> format not recognised. Look inside and tell the importer "
      "explicitly with --from.")
return 1

# ----- strawdi
def import_strawdi(src, w: Writer, default_class: int, min_px: int) -> None:
    """images + per-instance PNG masks -> boxes.

    Instances are the distinct non-zero values in the mask. If there is only
    one non-zero value the mask is binary, and connected components are used
    instead -- otherwise every berry in the frame would merge into one box
    spanning the whole plant, which is worse than no label at all.
    """
    np, Image = need_numpy()
    pairs = []
    for base, _d, files in os.walk(src):
        low = os.path.basename(base).lower()
        if low not in ("img", "image", "images"):
            continue
        for cand in ("label", "labels", "mask", "masks", "gt"):
            lab_dir = os.path.join(os.path.dirname(base), cand)
            if os.path.isdir(lab_dir):
                for f in sorted(files):
                    if not f.lower().endswith(IMG_EXT):
                        continue
                    for ext in IMG_EXT:
                        m = os.path.join(lab_dir, stem(f) + ext)
                        if os.path.exists(m):
                            pairs.append((os.path.join(base, f), m))
                            break
                break
    if not pairs:
        raise SystemExit(

```

```

        "no image/mask pairs found. Expected an 'img' directory beside a "
        "'label' (or mask/gt) directory. Run --inspect first.")

for img_path, mask_path in pairs:
    mask = np.asarray(Image.open(mask_path))
    if mask.ndim == 3:
        mask = mask[:, :, 0]
    H, W = mask.shape
    values = [v for v in np.unique(mask) if v != 0]
    boxes = []
    if len(values) > 1:
        regions = [(mask == v) for v in values]
    else:
        regions = list(_components(mask > 0, np))
    for reg in regions:
        ys, xs = np.where(reg)
        if xs.size < min_px:
            continue
        x0, x1, y0, y1 = xs.min(), xs.max(), ys.min(), ys.max()
        boxes.append((default_class,
                      (x0 + x1 + 1) / 2.0 / W, (y0 + y1 + 1) / 2.0 / H,
                      (x1 - x0 + 1) / W, (y1 - y0 + 1) / H))
    w.add(img_path, boxes)

def _components(binary, np):
    """4-connectivity components, iterative. Only used for binary masks."""
    seen = np.zeros_like(binary)
    H, Wd = binary.shape
    for y0 in range(H):
        for x0 in range(Wd):
            if not binary[y0, x0] or seen[y0, x0]:
                continue
            stack = [(y0, x0)]
            seen[y0, x0] = True
            comp = np.zeros_like(binary)
            while stack:
                y, x = stack.pop()
                comp[y, x] = True
                for dy, dx in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    ny, nx = y + dy, x + dx
                    if (0 <= ny < H and 0 <= nx < Wd
                        and binary[ny, nx] and not seen[ny, nx]):
                        seen[ny, nx] = True
                        stack.append((ny, nx))
            yield comp

# ----- yolo
def import_yolo(src, w: Writer, mapping: dict, ours: dict) -> None:
    names = None
    for base, _d, files in os.walk(src):
        for f in files:
            if f.lower() in ("data.yaml", "data.yml"):
                d = yaml.safe_load(open(os.path.join(base, f))) or {}
                n = d.get("names")
                if isinstance(n, dict):
                    names = {int(k): v for k, v in n.items()}
                elif isinstance(n, list):
                    names = dict(enumerate(n))
    if names is None:
        raise SystemExit(
            "no data.yaml with class names found, so the source class ids are "
            "undocumented and cannot be mapped safely. Open a label file, work "
            "out what each id means, and pass --map \"0=ripe_strawberry,...\" "
            "using the NUMBERS as source keys.")
    print(" source classes:", names)

    for img in find_images(src):
        parts = img.split(os.sep)
        if "labels" in parts:
            continue # do not import label images as data
        lab = None
        for cand in (img.rsplit(".", 1)[0] + ".txt",
                    img.replace(os.sep + "images" + os.sep,
                               os.sep + "labels" + os.sep,
                               1).rsplit(".", 1)[0] + ".txt"):
            if os.path.exists(cand):
                lab = cand
                break
        if lab is None:
            continue
        boxes = []
        for line in open(lab):
            p = line.split()
            if len(p) < 5:

```

```

        continue
    sid = int(float(p[0]))
    key = names.get(sid, str(sid))
    if key in mapping:
        dest = mapping[key]
    elif str(sid) in mapping:
        dest = mapping[str(sid)]
    else:
        w.dropped[key] += 1
        continue
    boxes.append((dest, *(float(v) for v in p[1:5])))
w.add(img, boxes)

# ----- COCO
def import_coco(src, w: Writer, mapping: dict) -> None:
    js = [os.path.join(b, f) for b, _d, fs in os.walk(src)
           for f in fs if f.lower().endswith(".json")]
    ann_file = None
    for j in js:
        try:
            d = json.load(open(j))
        except Exception:
            continue
        if isinstance(d, dict) and "annotations" in d and "images" in d:
            ann_file, data = j, d
            break
    if ann_file is None:
        raise SystemExit("no COCO json with 'images' and 'annotations' found")
    cats = {c["id"]: c["name"] for c in data.get("categories", [])}
    print(" source categories:", cats)
    imgs = {i["id"]: i for i in data["images"]}
    by_img: dict[int, list] = {}
    for a in data["annotations"]:
        by_img.setdefault(a["image_id"], []).append(a)

    root = os.path.dirname(ann_file)
    for iid, info in imgs.items():
        path = None
        for cand in (os.path.join(root, info["file_name"]),
                     os.path.join(root, "images", info["file_name"])):
            if os.path.exists(cand):
                path = cand
                break
        if path is None:
            continue
        W, H = float(info["width"]), float(info["height"])
        boxes = []
        for a in by_img.get(iid, []):
            key = cats.get(a["category_id"], str(a["category_id"]))
            if key not in mapping:
                w.dropped[key] += 1
                continue
            x, y, bw, bh = a["bbox"] # COCO: top-left x, y, w, h
            boxes.append((mapping[key], (x + bw / 2) / W, (y + bh / 2) / H,
                               bw / W, bh / H))
        w.add(path, boxes)

# ----- VOC
def import_voc(src, w: Writer, mapping: dict) -> None:
    import xml.etree.ElementTree as ET
    for img in find_images(src):
        xml = img.rsplit(".", 1)[0] + ".xml"
        if not os.path.exists(xml):
            continue
        root = ET.parse(xml).getroot()
        size = root.find("size")
        W = float(size.find("width").text)
        H = float(size.find("height").text)
        boxes = []
        for obj in root.findall("object"):
            key = obj.find("name").text.strip()
            if key not in mapping:
                w.dropped[key] += 1
                continue
            b = obj.find("bndbox")
            x0, y0 = float(b.find("xmin").text), float(b.find("ymin").text)
            x1, y1 = float(b.find("xmax").text), float(b.find("ymax").text)
            boxes.append((mapping[key], (x0 + x1) / 2 / W, (y0 + y1) / 2 / H,
                               (x1 - x0) / W, (y1 - y0) / H))
        w.add(img, boxes)

# ----- main
def main() -> int:

```

```

here = os.path.dirname(os.path.abspath(__file__))
ap = argparse.ArgumentParser(
    description=__doc__,
    formatter_class=argparse.RawDescriptionHelpFormatter)
ap.add_argument("--inspect", metavar="DIR",
    help="report what a downloaded folder contains and exit")
ap.add_argument("--src")
ap.add_argument("--out")
ap.add_argument("--from", dest="fmt",
    choices=["strawdi", "yolo", "coco", "voc"])
ap.add_argument("--map", help='e.g. "strawberry=ripe_strawberry, '
    'unripe=unripe_strawberry"')
ap.add_argument("--default-class", default=None,
    help="class for mask datasets that do not label ripeness")
ap.add_argument("--prefix", default=None,
    help="filename prefix; defaults to the source folder name")
ap.add_argument("--min-mask-px", type=int, default=12)
ap.add_argument("--classes", default=os.path.join(here, "classes.yaml"))
args = ap.parse_args()

if args.inspect:
    return inspect(os.path.expanduser(args.inspect))
if not (args.src and args.out and args.fmt):
    ap.error("--src, --out and --from are required (or use --inspect)")

ours = load_our_classes(args.classes)
by_name = {v: k for k, v in ours.items()}
src = os.path.expanduser(args.src)
out = os.path.expanduser(args.out)
prefix = args.prefix or os.path.basename(src.rstrip("/")) or "real"
w = Writer(out, prefix)

if args.fmt == "strawdi":
    if args.default_class is None:
        print("--default-class is required for mask datasets.\n"
            "They do not label ripeness: a mask is a strawberry, full "
            "stop. Which of our classes that becomes is your decision, "
            f"not the importer's. Choose from {sorted(by_name)}")
        return 2
    dc = (int(args.default_class) if args.default_class.isdigit()
        else by_name.get(args.default_class))
    if dc is None:
        print(f"unknown class {args.default_class!r}; "
            f"choose from {sorted(by_name)}")
        return 2
    import_strawdi(src, w, dc, args.min_mask_px)
    if ours[dc] == "ripe_strawberry":
        print("\nWARNING: everything in this set was imported as "
            "ripe_strawberry. If the images contain green berries, the "
            "model is now being taught that green berries are ripe -- "
            "which destroys the very distinction unripe_strawberry "
            "exists to make. Look at a sample before training.")
else:
    mapping = parse_map(args.map, ours)
    if mapping is None:
        print("--map is required for this format, and it is not "
            "guessable: a set's 'strawberry' may mean 'ripe' or 'any "
            "strawberry', and those are different classes for us. Run "
            "--inspect to see the source class names.")
        return 2
    if args.fmt == "yolo":
        import_yolo(src, w, mapping, ours)
    elif args.fmt == "coco":
        import_coco(src, w, mapping)
    else:
        import_voc(src, w, mapping)

print(f"\nwrote {out}")
print(f" {w.n_img} images, {w.n_box} boxes, {w.n_empty} with no object")
print(f" per class: "
    f"{ {ours.get(c, c): n for c, n in sorted(w.per_class.items()) } }")
if w.dropped:
    print(f" DROPPED unmapped source classes: {dict(w.dropped)}")
    print(" Those objects are now absent from the labels. If any of them "
        "IS fruit, every one is a false negative the model will be "
        "punished for finding. Map them or remove those images.")
if not w.n_img:
    print(" nothing imported -- run --inspect and check --from")
    return 1
print(f"\nNext: python3 ml/prepare_dataset.py --source {out}:real ...")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

ml/prepare_dataset.py

268 lines

```
#!/usr/bin/env python3
"""Merge every data source into one YOLO tree with honest splits.

WHY MERGING NEEDS A SCRIPT AND NOT A `cp`

Three things go wrong when datasets are combined by hand, and all three are
invisible afterwards -- they show up only as a model that scores well and
fails in the field.

1. LEAKAGE BETWEEN SPLITS. Frames from one capture session are highly
correlated: a robot at 0.2 m/s photographs the same plant twenty times.
Splitting those at random puts near-duplicates in both train and val, the
validation score climbs, and it measures nothing. This script splits BY
SOURCE GROUP, never by individual image, so a session lands wholly in one
split.

2. SYNTHETIC IMAGES IN THE TEST SET. A test set containing Gazebo renders
cannot tell you whether the model works on a real greenhouse -- which is
the only question. Sources are tagged `real` or `synthetic`, and by default
the test split is drawn from REAL sources only. The script refuses to
proceed without at least one real source unless you say so explicitly.

3. CLASS ID COLLISIONS. Every public dataset numbers its classes from 0. A
strawberry set's class 0 and a glass set's class 0 mean different things,
and concatenating them trains a model on a contradiction. Per-source class
remapping is mandatory here, not optional.

USAGE

python3 prepare_dataset.py \\\
  --source ~/youbot_datasets/twin_morning:synthetic \\\
  --source ~/downloads/strawdi_yolo:real:0=0 \\\
  --source ~/photos_serre:real \\\
  --out ~/strawberry_ds

A source is `PATH:KIND[:MAP]` where KIND is `real` or `synthetic` and MAP is
an optional comma-separated `old=new` class remap (unlisted classes are
dropped, loudly).
"""

from __future__ import annotations

import argparse
import os
import random
import shutil
import sys
from collections import Counter

import yaml

IMG_EXT = (".png", ".jpg", ".jpeg", ".bmp", ".ppm")

def load_classes(path: str) -> dict:
    with open(path) as fh:
        data = yaml.safe_load(fh)
    return {int(k): v for k, v in data["names"].items()}

def parse_source(spec: str):
    """`PATH:KIND[:MAP]` -> (path, kind, {old: new} or None)."""
    parts = spec.split(":")
    if len(parts) < 2:
        raise SystemExit(f"source {spec!r} needs PATH:KIND (real|synthetic)")
    path = os.path.expanduser(parts[0])
    kind = parts[1].lower()
    if kind not in ("real", "synthetic"):
        raise SystemExit(f"source {spec!r}: KIND must be real or synthetic")
    remap = None
    if len(parts) > 2 and parts[2].strip():
        remap = {}
        for pair in parts[2].split(","):
            old, new = pair.split("=")
            remap[int(old)] = int(new)
    return path, kind, remap

def find_pairs(root: str):
    """Every (image, label) pair under a YOLO-layout directory.

    Accepts both `/images` + `/labels` and a flat directory with
    the .txt beside the image, because the two conventions are equally common

```

```

and getting it wrong yields an empty dataset with no error.
"""
img_dir = os.path.join(root, "images")
lab_dir = os.path.join(root, "labels")
if not os.path.isdir(img_dir):
    img_dir = lab_dir = root
pairs = []
if not os.path.isdir(img_dir):
    return pairs
for name in sorted(os.listdir(img_dir)):
    if not name.lower().endswith(IMG_EXT):
        continue
    stem = os.path.splitext(name)[0]
    lab = os.path.join(lab_dir, stem + ".txt")
    pairs.append((os.path.join(img_dir, name),
                      lab if os.path.exists(lab) else None))
return pairs

def read_label(path, remap, dropped: Counter):
    """Read one YOLO label file, applying the class remap."""
    if path is None:
        return None # unlabelled: not the same as empty
    out = []
    with open(path) as fh:
        for line in fh:
            p = line.split()
            if len(p) < 5:
                continue
            cls = int(float(p[0]))
            if remap is not None:
                if cls not in remap:
                    dropped[cls] += 1
                    continue
                cls = remap[cls]
            out.append((cls, *(float(v) for v in p[1:5])))
    return out

def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--source", action="append", required=True,
                    help="PATH:KIND[:MAP], repeatable")
    ap.add_argument("--out", required=True)
    ap.add_argument("--classes", default=os.path.join(
        os.path.dirname(os.path.abspath(__file__)), "classes.yaml"))
    ap.add_argument("--val-frac", type=float, default=0.15)
    ap.add_argument("--test-frac", type=float, default=0.15)
    ap.add_argument("--seed", type=int, default=1)
    ap.add_argument("--allow-synthetic-test", action="store_true",
                    help="let synthetic frames into the test split. Only for "
                        "a sanity run -- the resulting score does not "
                        "predict field behaviour.")
    args = ap.parse_args()

    names = load_classes(args.classes)
    out = os.path.expanduser(args.out)
    rng = random.Random(args.seed)

    sources = [parse_source(s) for s in args.source]
    real = [s for s in sources if s[1] == "real"]
    if not real and not args.allow_synthetic_test:
        print("REFUSING: every source is synthetic, so the test set would be "
              "renders. A score on rendered images does not predict a real "
              "greenhouse. Add a real source, or pass --allow-synthetic-test "
              "if you know that is what you want.")
        return 2

    # ---- gather, keeping each source as one indivisible group ----
    groups = [] # (name, kind, [(img, label_rows)])
    dropped = Counter()
    for path, kind, remap in sources:
        pairs = find_pairs(path)
        if not pairs:
            print(f"WARNING: no images found under {path} -- skipping")
            continue
        items = [(img, read_label(lab, remap, dropped)) for img, lab in pairs]
        groups.append((os.path.basename(path.rstrip("/")) or path, kind, items))
        print(f" {path}: {len(items)} images ({kind})")
    if not groups:
        print("nothing to do: no source contained images")
        return 2
    if dropped:
        print(f" dropped labels with unmapped classes: {dict(dropped)}")

    # ---- split BY GROUP, test from real sources only ----

```

```

real_groups = [g for g in groups if g[1] == "real"]
syn_groups = [g for g in groups if g[1] == "synthetic"]
rng.shuffle(real_groups)

test_groups, val_groups, train_groups = [], [], []
if args.allow_synthetic_test and not real_groups:
    rng.shuffle(syn_groups)
    pool, syn_groups = syn_groups, []
else:
    pool = real_groups

total_real = sum(len(g[2]) for g in pool)
want_test = total_real * args.test_frac
want_val = total_real * args.val_frac
got_test = got_val = 0
for g in pool:
    if got_test < want_test:
        test_groups.append(g)
        got_test += len(g[2])
    elif got_val < want_val:
        val_groups.append(g)
        got_val += len(g[2])
    else:
        train_groups.append(g)
# Everything synthetic trains. It never validates and never tests.
train_groups += syn_groups

if not test_groups:
    print("WARNING: the test split is EMPTY. With only one real source "
          "there is nothing to hold out -- split it into sessions first.")

# ---- write -----
counts = {}
for split, gs in (("train", train_groups), ("val", val_groups),
                  ("test", test_groups)):
    idir = os.path.join(out, split, "images")
    ldir = os.path.join(out, split, "labels")
    os.makedirs(idir, exist_ok=True)
    os.makedirs(ldir, exist_ok=True)
    n_img = n_box = n_empty = n_unlabelled = 0
    per_class = Counter()
    for gname, _kind, items in gs:
        for img, rows in items:
            if rows is None:
                n_unlabelled += 1
                continue # never ship an unlabelled image
            stem = f"{gname}_{os.path.splitext(os.path.basename(img))[0]}"
            shutil.copy2(img, os.path.join(
                idir, stem + os.path.splitext(img)[1]))
            with open(os.path.join(ldir, stem + ".txt"), "w") as fh:
                for cls, cx, cy, w, h in rows:
                    fh.write(f"{cls} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}\n")
                    per_class[cls] += 1
            if not rows:
                n_empty += 1
            n_img += 1
            n_box += len(rows)
    counts[split] = {"images": n_img, "boxes": n_box,
                    "empty": n_empty, "unlabelled_skipped": n_unlabelled,
                    "per_class": {names.get(c, c): n
                                   for c, n in sorted(per_class.items())}}

data_yaml = {
    "path": out,
    "train": "train/images",
    "val": "val/images",
    "test": "test/images",
    "names": names,
}
with open(os.path.join(out, "data.yaml"), "w") as fh:
    yaml.safe_dump(data_yaml, fh, sort_keys=False, allow_unicode=True)
with open(os.path.join(out, "splits.yaml"), "w") as fh:
    yaml.safe_dump({
        "counts": counts,
        "train_sources": [g[0] for g in train_groups],
        "val_sources": [g[0] for g in val_groups],
        "test_sources": [g[0] for g in test_groups],
        "split_rule": "by source group, never by image: frames from one "
                      "session are near-duplicates and splitting them at "
                      "random leaks the validation set",
        "test_rule": ("real sources only"
                      if not args.allow_synthetic_test
                      else "SYNTHETIC ALLOWED -- score does not predict "
                           "field behaviour"),
    }, fh, sort_keys=False, allow_unicode=True)

print(f"\nwrote {out}/data.yaml")

```

```

for split, c in counts.items():
    print(f" {split:5s} {c['images']:5d} images, {c['boxes']:6d} boxes, "
          f"{c['empty']:4d} empty {c['per_class']}")
if counts["train"]["boxes"] and not counts["test"]["images"]:
    print("\nReminder: with no test split you can tune but you cannot "
          "report a number.")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

ml/augment_daylight.py

137 lines

```

#!/usr/bin/env python3
"""Widen the daylight range of a dataset, offline.

WHY A SEPARATE SCRIPT WHEN YOLO ALREADY AUGMENTS

Ultralytics' hsv_h shifts hue by a fraction of the colour wheel, uniformly and
symmetrically. That is a generic augmentation. What breaks this project is not
generic: it is the specific, one-directional shift produced by sunlight through
glass as the day ends. Light goes WARM -- the blue channel falls, the red rises
-- and every surface in the frame moves the same way at once. That is a
blackbody shift, not a hue rotation, and the colour-threshold detector fails
precisely because its G/R test cannot tell a warm scene from a red object.

So the training set gets copies of itself under simulated colour temperatures
from cold overcast to low evening sun. Labels are copied unchanged -- geometry
does not move -- which is the whole reason this can be done offline for free.

This is the augmentation that stands between a model trained on midday images
and a model that still works at 17 h. If you only ever capture the twin at one
lighting setting, run this.

python3 augment_daylight.py --dataset ~/strawberry_ds --split train

Do NOT run it on val or test. Augmenting the test set measures the
augmentation, not the model.
"""

from __future__ import annotations

import argparse
import os
import shutil
import sys

IMG_EXT = (".png", ".jpg", ".jpeg", ".bmp")

# Approximate linear RGB gains for a range of daylight colour temperatures,
# normalised so that ~6500 K (overcast noon) is neutral. Derived from the
# standard blackbody approximation; exact enough for augmentation, and the
# point is the DIRECTION and spread, not radiometric accuracy.
TEMPERATURES = {
    "overcast_7500K": (0.92, 0.97, 1.10),
    "noon_6500K": (1.00, 1.00, 1.00),
    "afternoon_5000K": (1.10, 1.00, 0.85),
    "golden_4000K": (1.22, 0.98, 0.70),
    "sunset_3000K": (1.38, 0.93, 0.55),
}

def apply_temperature(rgb, gains, exposure=1.0):
    """Multiply the channels, in a rough linear space, then clip.

    sRGB is gamma-encoded, so scaling it directly exaggerates the shift in the
    shadows. Decoding with gamma 2.2, scaling, and re-encoding keeps the shift
    where a real illuminant change puts it -- across the whole tone range
    rather than concentrated in the darks.
    """
    import numpy as np
    lin = (rgb.astype(np.float32) / 255.0) ** 2.2
    for c in range(3):
        lin[:, :, c] *= gains[c] * exposure
    out = np.clip(lin, 0.0, 1.0) ** (1.0 / 2.2)
    return (out * 255.0 + 0.5).astype(np.uint8)

def main() -> int:
    ap = argparse.ArgumentParser(
        description=__doc__,
        formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--dataset", required=True, help="the prepare_dataset.py tree")

```

```

ap.add_argument("--split", default="train", choices=["train"],
                help="train only, deliberately: augmenting val or test "
                    "measures the augmentation instead of the model")
ap.add_argument("--temperatures", default="afternoon_5000K, golden_4000K, sunset_3000K",
                help="comma-separated keys from TEMPERATURES")
ap.add_argument("--exposure-jitter", type=float, default=0.15,
                help="+/- fraction of brightness variation per copy")
ap.add_argument("--seed", type=int, default=1)
args = ap.parse_args()

try:
    import numpy as np
    from PIL import Image
except ImportError:
    print("needs pillow and numpy: pip install -r requirements.txt")
    return 2

rng = np.random.default_rng(args.seed)
root = os.path.expanduser(args.dataset)
idir = os.path.join(root, args.split, "images")
ldir = os.path.join(root, args.split, "labels")
if not os.path.isdir(idir):
    print(f"no {idir} -- run prepare_dataset.py first")
    return 2

wanted = [t.strip() for t in args.temperatures.split(",") if t.strip()]
for t in wanted:
    if t not in TEMPERATURES:
        print(f"unknown temperature {t!r}. Known: {list(TEMPERATURES)}")
        return 2

originals = [f for f in sorted(os.listdir(idir))
              if f.lower().endswith(IMG_EXT) and "_ct" not in f]
if not originals:
    print(f"no source images in {idir}")
    return 2

made = 0
for name in originals:
    stem, ext = os.path.splitext(name)
    lab = os.path.join(ldir, stem + ".txt")
    if not os.path.exists(lab):
        continue # never fabricate a label
    rgb = np.asarray(Image.open(os.path.join(idir, name)).convert("RGB"))
    for key in wanted:
        exposure = 1.0 + float(rng.uniform(-args.exposure_jitter,
                                           args.exposure_jitter))
        out = apply_temperature(rgb, TEMPERATURES[key], exposure)
        new_stem = f"{stem}_ct{key}"
        Image.fromarray(out).save(os.path.join(idir, new_stem + ext))
        # Labels are copied verbatim: an illuminant change moves no pixel.
        shutil.copy2(lab, os.path.join(ldir, new_stem + ".txt"))
    made += 1

print(f"{len(originals)} source images -> {made} colour-temperature copies "
      f"in {args.split}")
print("Labels were copied unchanged: changing the illuminant moves no "
      "geometry, which is why this is free.")
print("\nSanity check before training: open a few *_ctsunset_3000K images. "
      "If the plants now look red to YOU, that is exactly the condition "
      "the colour threshold fails in, and now the model has seen it.")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

ml/train.py

155 lines

```

#!/usr/bin/env python3
"""Train the strawberry / plant / glass detector.

```

Thin on purpose: Ultralytics does the work, and this file exists to pin the handful of settings that actually matter for THIS problem and to say why, so that nobody re-tunes them from a blog post six months from now.

THE TWO SETTINGS THAT MATTER MORE THAN THE MODEL SIZE

1. IMAGE SIZE. Strawberries are small objects. A 34 mm berry at 2 m subtends about 15 px in a 640 px frame with this camera's 1.2 rad field of view, and YOLO's coarsest head has stride 32 -- below roughly 20 px, recall falls off a cliff. Training at 960 px moves recall at range more than going from yolov8n to yolov8m does, and it is the first thing to raise if distant fruit is being missed.

2. HUE AUGMENTATION. Ultralytics defaults to `hsv_h = 0.015`, which is a very narrow band of colour shift. That default is wrong for this project specifically, because the exact failure being fixed IS a colour shift: the colour-threshold detector survives shade (illumination scale) and fails at 17 h (illumination colour temperature). Training with the default hue range on midday images reproduces that failure in a neural network -- a more expensive way to get the same bug. `hsv_h` is raised to 0.05 here.

WHAT THIS SCRIPT DELIBERATELY DOES NOT DO

It does not report a final number. Validation mAP is for choosing a checkpoint. The number that goes in the report comes from stage 8, on the held-out real test set, against the colour-threshold baseline on identical images:

```
python3 predict_to_labels.py --weights <best.pt> --images <test/images> --out <preds>
ros2 run youbot_commissioning stage8_detector -- \
  --labels <test/labels> --predict <preds> --baseline <colour_preds>
"""
```

```
from __future__ import annotations
```

```
import argparse
import os
import sys
```

```
import yaml
```

Chosen for this problem, with the reasoning attached. Anything not listed here is an Ultralytics default and was left alone on purpose.

```
TUNED = {
    # --- the colour-robustness block -----
    "hsv_h": 0.05,      # 3.3x the default. See the header: the failure mode
                        # being fixed is a hue shift, so the training data must
                        # contain hue shifts.
    "hsv_s": 0.7,       # saturation: wet fruit and overcast light
    "hsv_v": 0.5,       # brightness: sunlit vs shaded row, same aisle

    # --- the small-object block -----
    "mosaic": 1.0,      # four images per sample: the single most effective
                        # augmentation for small objects, because it puts them
                        # at more scales and positions than the data contains
    "close_mosaic": 10, # off for the last 10 epochs: mosaic distorts object
                        # statistics, and the final epochs should see real
                        # framing
    "scale": 0.5,       # +/-50 % zoom: stands in for the 0.3-2.5 m range
    "copy_paste": 0.0,  # needs masks; enable if you annotate plants as masks

    # --- geometry -----
    "fliplr": 0.5,      # the robot drives an aisle in both directions
    "flipud": 0.0,      # it never drives upside down; this would be noise
    "degrees": 5.0,     # small: the camera is on a levelled base
    "perspective": 0.0,

    # --- optimisation -----
    "optimizer": "auto",
    "patience": 30,     # early stop; small agricultural sets plateau early
    "cos_lr": True,
}
```

```
def main() -> int:
    ap = argparse.ArgumentParser(
        description=__doc__,
        formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--data", required=True, help="data.yaml from prepare_dataset.py")
    ap.add_argument("--model", default="yolov8s",
                    help="yolov8n|s|m|l, or a .pt to fine-tune from")
    ap.add_argument("--imgsz", type=int, default=960,
                    help="960 or 1280. Do NOT drop to 640 for this problem.")
    ap.add_argument("--epochs", type=int, default=150)
    ap.add_argument("--batch", type=int, default=8,
                    help="-1 lets Ultralytics pick from free VRAM")
    ap.add_argument("--device", default="",
                    help="'0' for the first GPU, 'cpu' to suffer")
    ap.add_argument("--name", default="strawberry")
    ap.add_argument("--freeze", type=int, default=0,
                    help="freeze the first N layers. Use ~10 when "
                    "fine-tuning a synthetic-trained model on a few "
                    "hundred real images, so the backbone is not "
                    "destroyed by a small set.")
    ap.add_argument("--dry-run", action="store_true",
                    help="print the resolved configuration and exit")
    args = ap.parse_args()

    data_path = os.path.expanduser(args.data)
```

```

if not os.path.exists(data_path):
    print(f"No data.yaml at {data_path} -- run prepare_dataset.py first")
    return 2
with open(data_path) as fh:
    data = yaml.safe_load(fh)

cfg = dict(TUNED)
cfg.update({
    "data": data_path,
    "imgsz": args.imgsz,
    "epochs": args.epochs,
    "batch": args.batch,
    "name": args.name,
    "freeze": args.freeze or None,
})
if args.device:
    cfg["device"] = args.device

print("classes:", data.get("names"))
print("resolved training configuration:")
for k in sorted(cfg):
    print(f"  {k:14s} {cfg[k]}")
if args.imgsz < 960:
    print("\nWARNING: imgsz < 960. A berry at 2 m is ~15 px at 640, below "
          "the stride-32 head's useful size. Expect recall at range to "
          "collapse and do not blame the model size for it.")
if args.dry_run:
    return 0

try:
    from ultralytics import YOLO
except ImportError:
    print("\n ultralytics is not installed: pip install -r requirements.txt")
    return 2

model = YOLO(args.model if args.model.endswith(".pt")
              else f"{args.model}.pt")
model.train(**cfg)

print("\nTraining finished. The validation mAP above is for PICKING a "
      "checkpoint, not for reporting.")
print("The number that goes in the report comes from stage 8, on the "
      "held-out REAL test split, against the colour baseline:")
print(" python3 predict_to_labels.py --weights runs/detect/"
      f"{args.name}/weights/best.pt --images <test/images> --out <preds>")
print(" ros2 run youbot_commissioning stage8_detector -- "
      "--labels <test/labels> --predict <preds> --baseline <colour_preds>")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

ml/predict_to_labels.py

211 lines

```

#!/usr/bin/env python3
"""Run a model over a folder and write YOLO label files stage 8 can score.

This is the bridge between training and measurement, and it exists so that the
learned model and the colour-threshold baseline are scored by the SAME code on
the SAME images. Reporting a model's own validation mAP against a number
someone quoted for the old detector compares nothing.

Two modes:

    --weights best.pt          the learned model
    --baseline-colour          the CURRENT colour-threshold detector, reimplemented
                              here exactly as strawberry_detector applies it

The second is the point. Without it there is no honest comparison: the old
detector was measured in simulation, on different images, by a different
scorer. Run both over the held-out real test set and the difference means
something.

python3 predict_to_labels.py --weights best.pt --images test/images --out preds/yolo
python3 predict_to_labels.py --baseline-colour --images test/images --out preds/colour

ros2 run youbot_commissioning stage8_detector -- \
  --labels test/labels --predict preds/yolo --baseline preds/colour

WRITING AN EMPTY FILE IS MANDATORY
An image with no detection gets an EMPTY .txt, never no file. stage8 counts a
missing prediction file as "the model was not run here" and warns; an empty one
is a real, scoreable claim of "nothing in this image". Confusing the two moves

```

```

false negatives around silently.
"""

from __future__ import annotations

import argparse
import os
import sys

IMG_EXT = (".png", ".jpg", ".jpeg", ".bmp")

# -----
# The colour-threshold baseline, transcribed from
# youbot_control/youbot_control/lib/vision.py so the comparison is against
# the detector that is actually running on the robot, not a rough equivalent.
# -----
def colour_boxes(rgb, value_min=60, red_ratio=0.33, min_pixels=6):
    """Ratio-based red segmentation + connected components -> boxes.

    This is the method being replaced. Its known weakness, and the reason the
    learned model exists, is that the ratio is invariant to illumination
    SCALE but not to illumination COLOUR: warm light lowers G/R across the
    whole scene and everything passes.
    """
    import numpy as np
    from scipy import ndimage # optional; falls back below

    r = rgb[:, :, 0].astype(np.int16)
    g = rgb[:, :, 1].astype(np.int16)
    b = rgb[:, :, 2].astype(np.int16)
    limit = red_ratio * r
    mask = (r > value_min) & (r >= g) & (r >= b) & (g < limit) & (b < limit)

    lab, n = ndimage.label(mask)
    h, w = mask.shape
    out = []
    for i in range(1, n + 1):
        ys, xs = np.where(lab == i)
        if ys.size < min_pixels:
            continue
        x0, x1 = xs.min(), xs.max()
        y0, y1 = ys.min(), ys.max()
        out.append((0,
                    ((x0 + x1) / 2.0 + 0.5) / w,
                    ((y0 + y1) / 2.0 + 0.5) / h,
                    (x1 - x0 + 1) / w,
                    (y1 - y0 + 1) / h,
                    1.0))

    return out

def colour_boxes_noscopy(rgb, value_min=60, red_ratio=0.33, min_pixels=6):
    """Same, with an explicit flood fill, for machines without scipy."""
    import numpy as np

    r = rgb[:, :, 0].astype(np.int16)
    g = rgb[:, :, 1].astype(np.int16)
    b = rgb[:, :, 2].astype(np.int16)
    limit = red_ratio * r
    mask = (r > value_min) & (r >= g) & (r >= b) & (g < limit) & (b < limit)

    h, w = mask.shape
    seen = np.zeros_like(mask)
    out = []
    for y0 in range(h):
        for x0 in range(w):
            if not mask[y0, x0] or seen[y0, x0]:
                continue
            stack = [(y0, x0)]
            seen[y0, x0] = True
            xs, ys = [], []
            while stack:
                y, x = stack.pop()
                xs.append(x)
                ys.append(y)
                for dy, dx in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    ny, nx = y + dy, x + dx
                    if (0 <= ny < h and 0 <= nx < w
                        and mask[ny, nx] and not seen[ny, nx]):
                        seen[ny, nx] = True
                        stack.append((ny, nx))
            if len(xs) < min_pixels:
                continue
            out.append((0,
                        (min(xs) + max(xs) + 1) / 2.0 / w,

```

```

        (min(ys) + max(ys) + 1) / 2.0 / h,
        (max(xs) - min(xs) + 1) / w,
        (max(ys) - min(ys) + 1) / h,
        1.0))

    return out

def load_rgb(path):
    from PIL import Image
    import numpy as np
    return np.asarray(Image.open(path).convert("RGB"))

def main() -> int:
    ap = argparse.ArgumentParser(
        description=__doc__,
        formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--images", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--weights", default=None)
    ap.add_argument("--baseline-colour", action="store_true",
        help="run the CURRENT colour threshold instead of a model")
    ap.add_argument("--conf", type=float, default=0.25)
    ap.add_argument("--imgsz", type=int, default=960)
    ap.add_argument("--device", default="")
    args = ap.parse_args()

    if bool(args.weights) == bool(args.baseline_colour):
        print("give exactly one of --weights or --baseline-colour")
        return 2

    img_dir = os.path.expanduser(args.images)
    out_dir = os.path.expanduser(args.out)
    os.makedirs(out_dir, exist_ok=True)
    images = [f for f in sorted(os.listdir(img_dir))
        if f.lower().endswith(IMG_EXT)]
    if not images:
        print(f"no images under {img_dir}")
        return 2

    written = boxes = empty = 0

    if args.baseline_colour:
        try:
            from scipy import ndimage # noqa: F401
            fn = colour_boxes
        except ImportError:
            print("scipy not found -- using the slower explicit flood fill")
            fn = colour_boxes_noscopy
        for name in images:
            rgb = load_rgb(os.path.join(img_dir, name))
            dets = fn(rgb)
            stem = os.path.splitext(name)[0]
            with open(os.path.join(out_dir, stem + ".txt"), "w") as fh:
                for cls, cx, cy, w, h, conf in dets:
                    fh.write(f"{cls} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f} "
                        f"{conf:.4f}\n")
            written += 1
            boxes += len(dets)
            empty += 1 if not dets else 0
    else:
        try:
            from ultralytics import YOLO
        except ImportError:
            print("ultralytics is not installed: pip install -r requirements.txt")
            return 2
        model = YOLO(os.path.expanduser(args.weights))
        for name in images:
            res = model.predict(os.path.join(img_dir, name),
                imgsz=args.imgsz, conf=args.conf,
                device=args.device or None, verbose=False)[0]
            stem = os.path.splitext(name)[0]
            n = 0
            with open(os.path.join(out_dir, stem + ".txt"), "w") as fh:
                for b in res.boxes:
                    cls = int(b.cls.item())
                    cx, cy, bw, bh = b.xywhn[0].tolist()
                    fh.write(f"{cls} {cx:.6f} {cy:.6f} {bw:.6f} {bh:.6f} "
                        f"{float(b.conf.item()):.4f}\n")
                    n += 1
            written += 1
            boxes += n
            empty += 1 if n == 0 else 0

    print(f"wrote {written} label files to {out_dir}: {boxes} boxes, "
        f"{empty} images with no detection (empty files, not missing ones)")

```

```
print("\nNow score BOTH against the same ground truth:")
print("  ros2 run youbot_commissioning stage8_detector -- \\")
print("    --labels <test/labels> --predict <yolo preds> "
      "--baseline <colour preds>")
return 0

if __name__ == "__main__":
    sys.exit(main())
```